

UML Notation:

In UML notation, you can represent the database schema with a class diagram. Each table becomes a class, and relationships between tables are represented as associations. Attributes of each class correspond to the table columns. Arrows and multiplicity indicators indicate the nature of the relationships.

For example, you would have classes representing "Employee," "Branch," "Employee_Order," "Employee_Projects," and "Dependent," with associations and cardinalities representing the relationships between them.

This project follows best practices for database design, PHP coding, and user interface design to create a comprehensive Employee Management System. It allows for efficient data storage, retrieval, and presentation while maintaining data integrity through normalization and validation.

Database Structure

The database for this Employee Management System consists of several tables to store information about employees, their orders, projects, and dependents. Here's an overview of the key tables and their relationships:

1. Employee Table:

- Fields: EmployeeID (Primary Key), Name, Residence, Salary, DateOfBirth, BranchID (Foreign Key to Branch Table).
- Represents information about employees, including their personal details.

2. Branch Table:

- Fields: BranchID (Primary Key), BranchName, BranchIdentification, PhoneNumber.
- Stores information about different branches of the company.

3. Employee_Order Table:

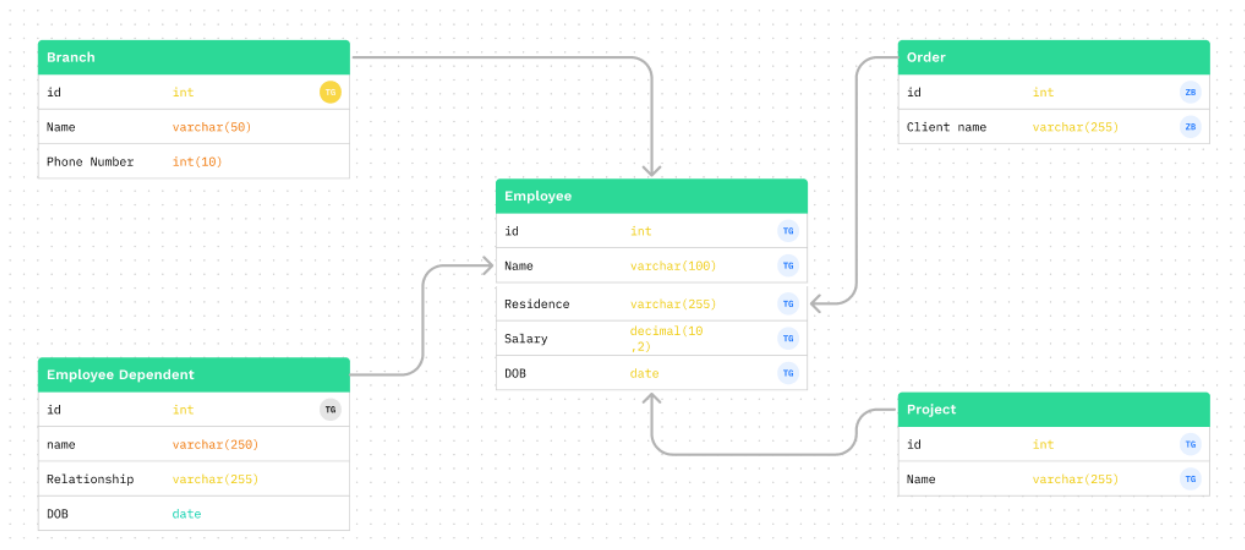
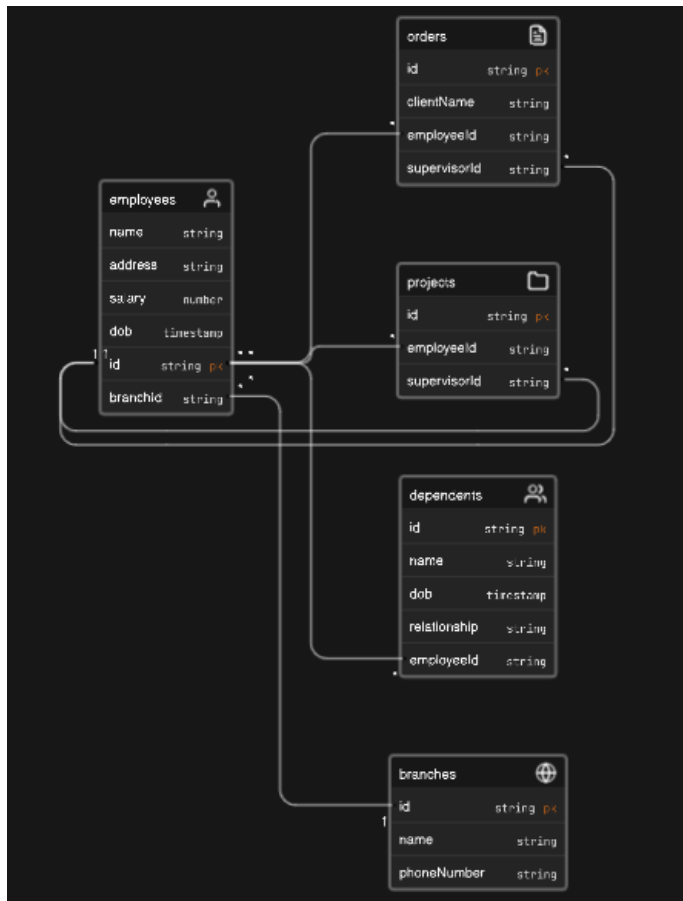
- Fields: OrderID (Primary Key), OrderIdentifier, ClientName, EmployeeID (Foreign Key to Employee Table).
- Records orders made by clients and associates them with employees.

4. Employee_Projects Table:

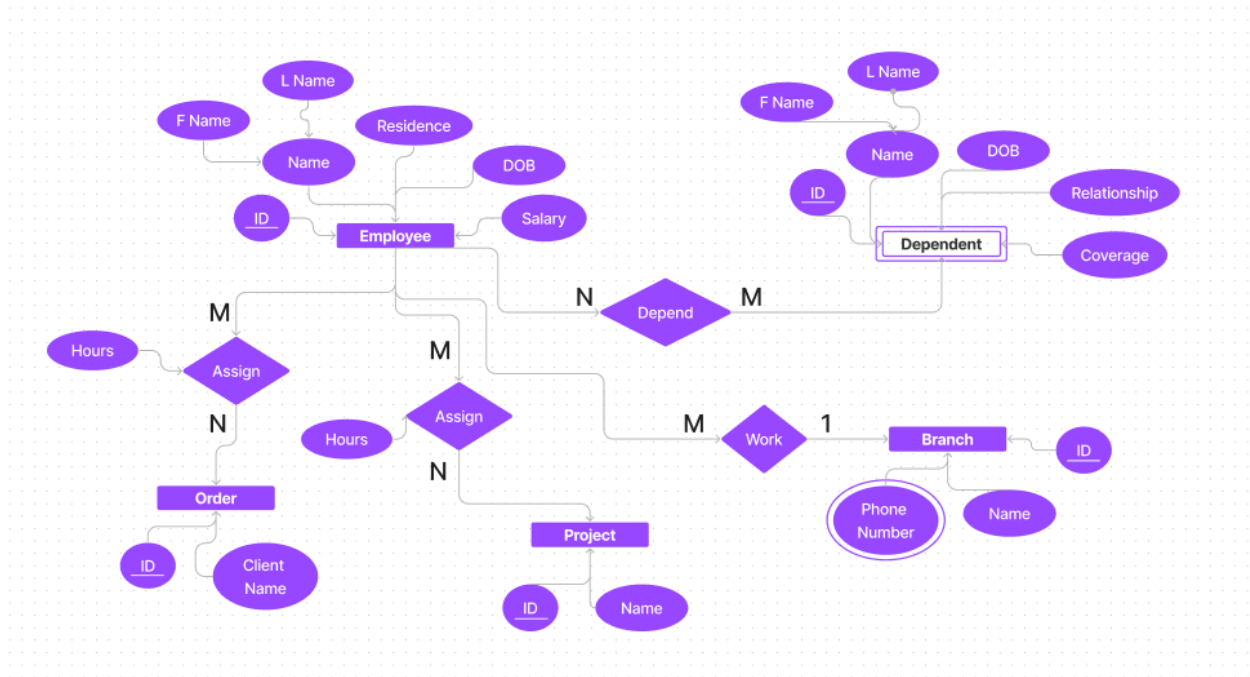
- Fields: ProjectID (Primary Key), ProjectIdentifier, EmployeeID (Foreign Key to Employee Table).
- Stores information about projects and associates them with employees.

5. Dependent Table:

- Fields: DependentID (Primary Key), Name, DateOfBirth, Relationship, EmployeeID (Foreign Key to Employee Table).
- Contains data about employee dependents for insurance purposes.



Data Base Idea



Data Base EERD Using UML notation.

Database Normalization

The project follows principles of database normalization to reduce data redundancy and maintain data integrity. Key normalizations include:

- First Normal Form (1NF): Each table contains only atomic (indivisible) data values without repeating groups or arrays.
- Second Normal Form (2NF): All non-key attributes are functionally dependent on the entire primary key.
- Third Normal Form (3NF): There is no transitive dependency between non-key attributes.
- Relationships between tables are defined using foreign keys to ensure data consistency and integrity.

PHP Code

The PHP code is used to interact with the database and display data to users. It includes:

- **Database Connection:** PHP code establishes a connection to the MySQL database using the MySQL extension.
- **Data Retrieval:** SQL queries are used to retrieve data from the database tables.
- **HTML Rendering:** The retrieved data is displayed in HTML format using Bootstrap for styling.
- **Error Handling:** Appropriate error handling is implemented to ensure a smooth user experience

Used Functions:

- `mysqli_connect()`: Function to establish a connection to the MySQL database.
- `mysqli_query()`: Function to execute SQL queries.
- `mysqli_prepare()`: Function to prepare SQL statements for execution.
- `mysqli_stmt_bind_param()`: Function to bind parameters to a prepared statement.
- `mysqli_fetch_assoc()`: Function to fetch rows from the result set as associative arrays.

How We Started this Project

Step 1: Project Planning and Setup

- **Objective:** Develop a web-based Employee Management System using PHP and MySQL to manage employees, branches, orders, projects, and dependents.
- **Tools:** You will need a text editor or integrated development environment (IDE), a web server (Apache), PHP, MySQL, and a browser.

Step 2: Environment Setup

1. **Install a Text Editor or IDE:** Download and install a code editor or IDE suitable for PHP development, such as Visual Studio Code
2. **Set Up a Local Web Server:** Choose a local development server solution like XAMPP to run PHP scripts and MySQL databases. Install it and start the server.

Step 3: Database Design

1. **Database Schema:** Create a database schema with tables for employees, branches, orders, projects, and dependents. Define relationships using foreign keys.
2. **Normalization:** Apply normalization techniques (1NF, 2NF, 3NF) to ensure efficient data storage and maintain data integrity.
3. **UML Notation:** Create a UML class diagram to represent the database schema, tables, and their relationships. Use rectangles for tables, lines with arrows for relationships, and labels for fields.

Step 4: Create Database Tables

1. **SQL Scripts:** Write SQL scripts to create the database tables based on your schema. Execute these scripts in your MySQL database.

```
1 CREATE TABLE Branch (
2   BranchName VARCHAR(255) PRIMARY KEY,
3   BranchID INT,
4   PhoneNumber VARCHAR(20)
5 );
6
7 CREATE TABLE Employee (
8   EmployeeID INT PRIMARY KEY,
9   Name VARCHAR(255),
10  Residence VARCHAR(255),
11  Salary DECIMAL(10, 2),
12  DateOfBirth DATE
13 );
14
15 CREATE TABLE Order (
16   OrderID INT PRIMARY KEY,
17   ClientName VARCHAR(255)
18 );
19
20 CREATE TABLE Dependent (
21   DependentID INT PRIMARY KEY,
22   Name VARCHAR(255),
23   DateOfBirth DATE,
24   Relationship VARCHAR(50)
25 );
26
27 CREATE TABLE Project (
28   ProjectID INT PRIMARY KEY,
29   ProjectName VARCHAR(255)
30 );
31
32 CREATE TABLE Employee_Order (
33   EmployeeID INT,
34   OrderID INT,
35   HourlyRate DECIMAL(10, 2),
36   PRIMARY KEY (EmployeeID, OrderID),
37   FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID),
38   FOREIGN KEY (OrderID) REFERENCES Order (OrderID)
39 );
40
41 CREATE TABLE Employee_Project (
42   EmployeeID INT,
43   ProjectID INT,
44   HourlyRate DECIMAL(10, 2),
45   PRIMARY KEY (EmployeeID, ProjectID),
46   FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID),
47   FOREIGN KEY (ProjectID) REFERENCES Project (ProjectID)
48 );
49
50 CREATE TABLE Manager (
51   BranchName VARCHAR(255),
52   EmployeeID INT,
53   PRIMARY KEY (BranchName),
54   FOREIGN KEY (BranchName) REFERENCES Branch (BranchName),
55   FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID)
56 );
57
58 CREATE TABLE NonDependent (
59   EmployeeID INT,
60   DependentID INT,
61   PRIMARY KEY (EmployeeID, DependentID),
62   FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID),
63   FOREIGN KEY (DependentID) REFERENCES Dependent (DependentID)
64 );
65
```

SQL Code

Step 5: Develop CRUD Operations

Branch Table:

- **Create Operation:** Develop PHP code to add new branch records to the "Branch" table. Use the `INSERT` statement to insert data.
- **Read Operation:** Create PHP code to retrieve and display branch records from the "Branch" table using the `SELECT` statement.
- **Update Operation:** Write PHP code to update existing branch records in the "Branch" table using the `UPDATE` statement.
- **Delete Operation:** Implement PHP code to delete branch records from the "Branch" table using the `DELETE` statement.

Step 6: Forms and User Interface

1. **HTML Forms:** Design HTML forms for adding, editing, and deleting branches. Include input fields for branch name, identification, phone number, etc.
2. **Bootstrap Styling:** Use Bootstrap classes for styling your forms and tables to create a user-friendly interface.

Step 7: Include PHP Files

1. **Modularization:** Break your PHP code into separate files for better organization and code reuse.
2. **Include Statements:** Use PHP's `include` statements to include common elements such as database connection code, headers, footers, and other reusable components.

Step 8: Testing and Debugging

1. **Testing:** Continuously test your project by adding, updating, and deleting branch records through the web forms. Check for errors and unexpected behavior.
2. **Debugging:** Use debugging tools like Xdebug and check error logs for issues. Fix any bugs and ensure smooth operation.

Step 9: Expand and Enhance

1. **Extend Functionality:** Add CRUD operations and user interfaces for other entities like employees, orders, projects, and dependents.
2. **Validation:** Implement input validation to ensure data integrity and security.

Step 10: Documentation and Deployment (Optional)

1. **Documentation:** Create documentation for your project, including a user manual and technical documentation.
2. **Deployment:** When ready, deploy your Employee Management System on a live web server for real-world usage.

Step 11: Version Control (Optional)

1. **Git Repository:** Consider setting up a Git repository to track changes, collaborate with team members, and manage code versions.

Step 12: Finalization and Submission

1. **Review and Finalization:** Review your project for completeness, functionality, and code quality.
2. **Submission:** If this project is for an assignment or coursework, submit your project following the guidelines provided by your instructor.

Step 13: Future Development (Optional)

1. **Ongoing Maintenance:** Continue maintaining and improving your Employee Management System based on feedback and evolving requirements.
2. **Advanced Features:** Explore adding advanced features such as user authentication, data visualization, and reporting.

Remember that this assignment guides you through the process of creating a basic Employee Management System. You can expand and enhance it based on your specific needs and requirements as you gain more experience in PHP development.

How to Start PHP web development

Set Up a Local Development Environment

1. **Install a Text Editor or Integrated Development Environment (IDE):** You can use text editors like Visual Studio Code, Sublime Text, or IDEs like PhpStorm for PHP development.
2. **Install a Local Web Server:** To run PHP scripts, you'll need a web server like Apache, Nginx, or use a package like XAMPP, WAMP, MAMP (for macOS), or LAMP (for Linux) that includes Apache, MySQL, and PHP. Download and install the one that suits your operating system.

Step 2: Create a Project Directory

1. **Create a Project Folder:** Choose a location on your computer where you want to store your PHP project. Create a folder with a meaningful name, e.g., "my-php-project."
2. **Create Subdirectories:** Inside your project folder, create subdirectories to organize your files. Common directories include "css" for stylesheets, "js" for JavaScript files, and "img" for images.

Step 3: Configure Your Web Server

1. **Apache Configuration (XAMPP, WAMP, etc.):** If you're using XAMPP or similar packages, open their control panel and start the Apache server. No additional configuration is usually required.
2. **Manual Apache Configuration:** If you're using Apache manually, configure the virtual host settings in the Apache configuration file (`httpd.conf` or `httpd-vhosts.conf`) to point to your project folder. Restart Apache after making changes.
3. **Nginx Configuration:** If you're using Nginx, create an Nginx server block (similar to Apache's virtual host) and configure it to serve your project files. Restart Nginx after making changes.

Step 4: Create and Write PHP Files

1. **Create a PHP File:** Inside your project folder, create a new PHP file with a ".php" extension. You can use any text editor or IDE to create and edit PHP files.
2. **Write PHP Code:** Start writing your PHP code in the PHP file. You can mix HTML and PHP within the same file. For example:

```
php
2. <?php
3. // PHP code goes here
4. echo "Hello, World!";
5. ?>
6.
```

Step 5: Access Your PHP Project

1. **Open a Web Browser:** Open your web browser (e.g., Chrome, Firefox).
2. **Access Your Localhost:** In the browser's address bar, type `http://localhost/my-php-project/` (replace "my-php-project" with the actual name of your project folder).
3. **View PHP Output:** You should see the output of your PHP script in the browser.

Step 6: Develop and Test Your PHP Project

1. **Continue Coding:** Keep developing your PHP project by adding more PHP files, creating forms, connecting to databases, and integrating with other technologies like JavaScript and CSS.
2. **Test Your Project:** Continuously test your PHP code by refreshing your browser or using tools like Postman for API testing.

Step 7: Database Setup (If Required)

1. **Install a Database:** If your project requires a database, install a database server (e.g., MySQL, PostgreSQL) and create a database for your project.
2. **Connect PHP to the Database:** Use PHP functions like `mysqli` or PDO to establish a connection to your database and perform database operations.

PHP `include` Statement

The `include` statement in PHP is used to include the content of another PHP file into the current PHP script. It allows you to modularize your code by separating it into multiple files, making your code more organized, maintainable, and easier to reuse.

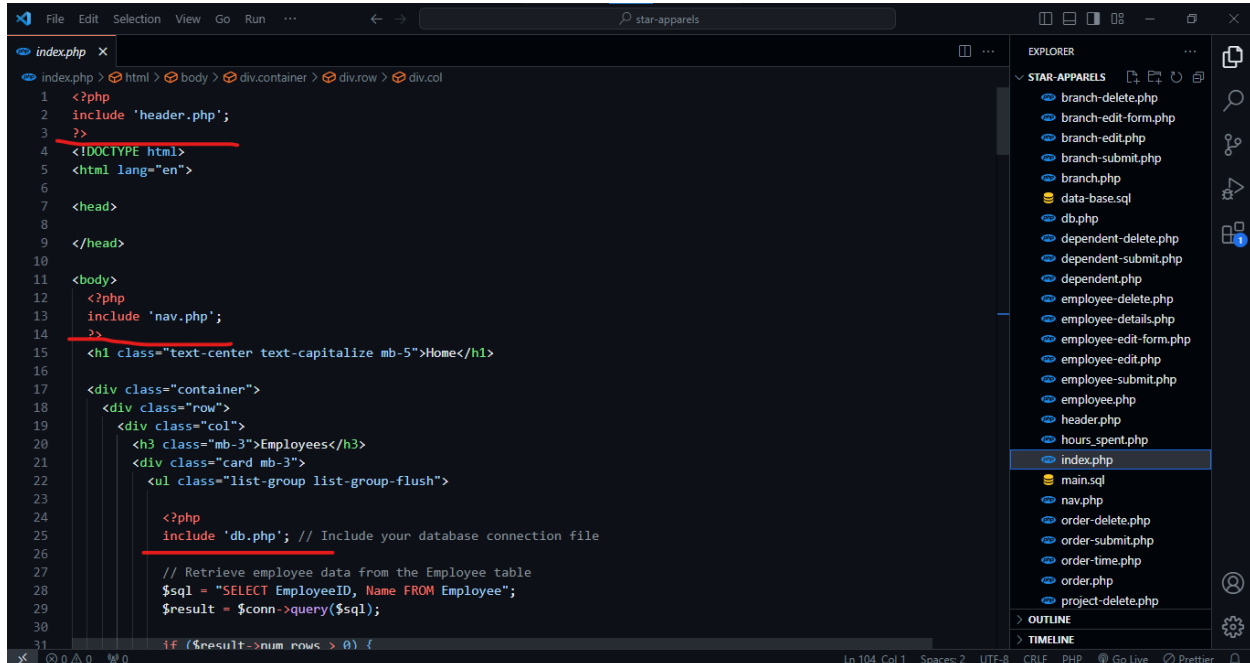
Syntax:

```
php
include 'filename.php';
```

Use cases of PHP `include`:

1. **Modularization and Code Reusability:** One of the primary use cases of `include` is to break your code into smaller, manageable parts. You can place reusable functions, headers, footers, or other pieces of code in separate files and include them where needed.

2. **Template Systems:** In web development, you can use `include` to include common elements like headers, footers, sidebars, or navigation menus across multiple web pages. This ensures consistency in design and functionality.
3. **Configuration Files:** You can store configuration settings, constants, or database connection details in separate PHP files. By including these files in your scripts, you centralize configuration management and make it easier to update settings globally.
4. **Library Inclusion:** When working with external libraries or frameworks, you can use `include` to load library files into your PHP scripts. This allows you to use the library's functions and classes in your code.



```
index.php X
index.php > html > body > div.container > div.row > div.col
1  <?php
2  include 'header.php';
3  >
4  <!DOCTYPE html>
5  <html lang="en">
6
7  <head>
8
9  </head>
10
11 <body>
12   <?php
13   include 'nav.php';
14   >
15   <h1 class="text-center text-capitalize mb-5">Home</h1>
16
17   <div class="container">
18     <div class="row">
19       <div class="col">
20         <h3 class="mb-3">Employees</h3>
21         <div class="card mb-3">
22           <ul class="list-group list-group-flush">
23
24             <?php
25             include 'db.php'; // Include your database connection file
26
27             // Retrieve employee data from the Employee table
28             $sql = "SELECT EmployeeID, Name FROM Employee";
29             $result = $conn->query($sql);
30
31             if ($result->num_rows > 0) {
```

EXPLORER

- STAR-APPARELS
 - branch-delete.php
 - branch-edit-form.php
 - branch-edit.php
 - branch-submit.php
 - branch.php
 - data-base.sql
 - db.php
 - dependent-delete.php
 - dependent-submit.php
 - dependent.php
 - employee-delete.php
 - employee-details.php
 - employee-edit-form.php
 - employee-edit.php
 - employee-submit.php
 - employee.php
 - header.php
 - hours_spent.php
 - index.php
 - main.sql
 - nav.php
 - order-delete.php
 - order-submit.php
 - order-time.php
 - order.php
 - project-delete.php
- OUTLINE
- TIMELINE

Why `include` Is Used in the Above Project:

In the Employee Management System project described earlier, the use of the `include` statement can be beneficial for several reasons:

1. **Code Organization:** The project likely consists of multiple PHP files, each responsible for different functionalities such as employee management, branch management, order management, and more. By using `include`, you can separate these functionalities into individual PHP files for better code organization.
2. **Reusability:** Common elements like database connection handling, authentication checks, or header and footer sections can be placed in separate PHP files and included wherever necessary across various pages of the project. This promotes code reuse and maintains consistency.
3. **Readability and Maintainability:** Smaller, modularized code files are easier to read, understand, and maintain. When you need to make changes or fix issues in a specific part of the project, you can quickly locate and update the relevant file without affecting other parts of the application.

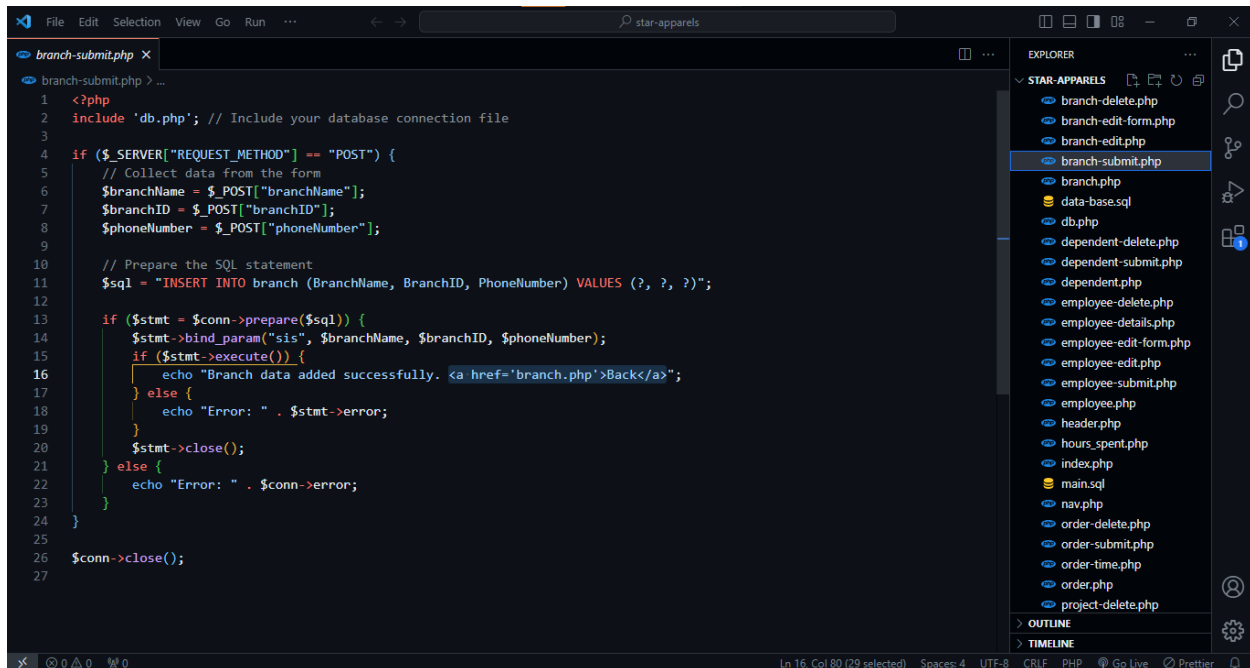
4. **Collaboration:** In a team-based project, different team members can work on separate modules of the application. Using `include` allows multiple developers to work on different parts of the project simultaneously, as long as they adhere to predefined interfaces and conventions.

For example, in the project, you might have separate PHP files for database connection (`db.php`) and more.

Create Branch CRUD Operations

1. Create (Add) Operation:

The "Create" operation involves adding new branch records to the "Branch" table.



```
1 <?php
2 include 'db.php'; // Include your database connection file
3
4 if ($_SERVER["REQUEST_METHOD"] == "POST") {
5     // Collect data from the form
6     $branchName = $_POST["branchName"];
7     $branchID = $_POST["branchID"];
8     $phoneNumber = $_POST["phoneNumber"];
9
10    // Prepare the SQL statement
11    $sql = "INSERT INTO branch (BranchName, BranchID, PhoneNumber) VALUES (?, ?, ?)";
12
13    if ($stmt = $conn->prepare($sql)) {
14        $stmt->bind_param("sis", $branchName, $branchID, $phoneNumber);
15        if ($stmt->execute()) {
16            echo "Branch data added successfully. <a href='branch.php'>Back</a>";
17        } else {
18            echo "Error: " . $stmt->error;
19        }
20        $stmt->close();
21    } else {
22        echo "Error: " . $conn->error;
23    }
24 }
25
26 $conn->close();
27
```

STAR APPARELS

HomeBranchEmployeeOrderProjectDependent

Branches

Input Data

ID

Name

Phone Number

Submit

All data

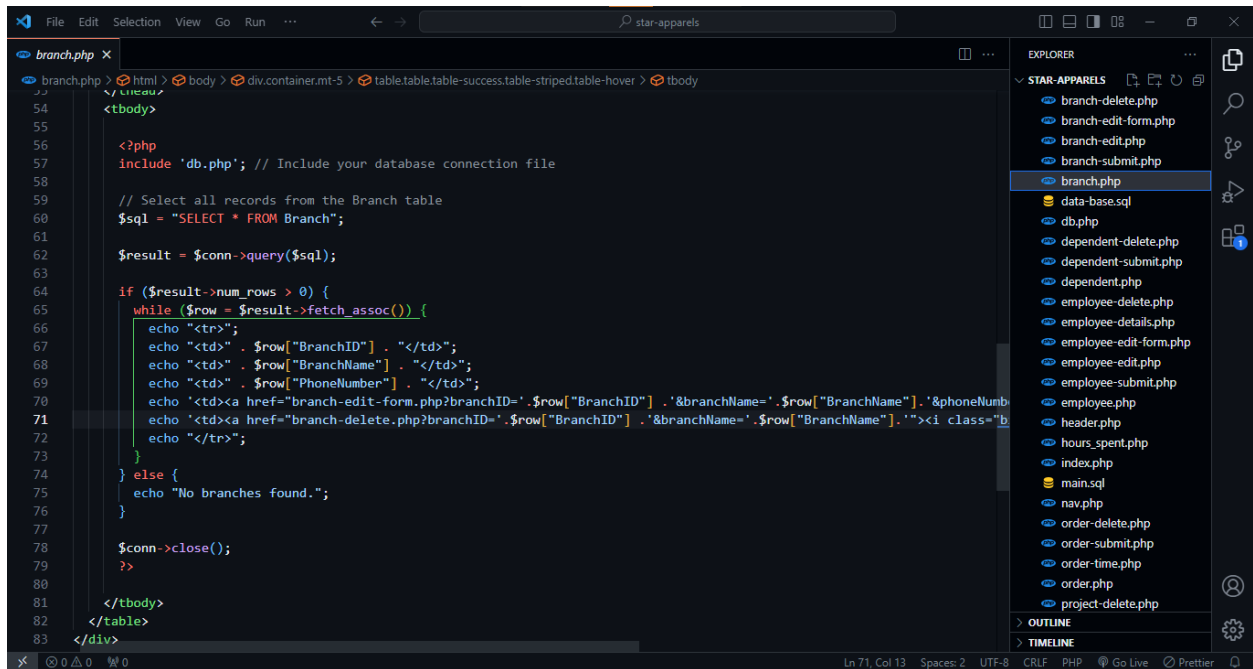
ID	Name	Phone Number	Edit	Delete
1	Colombo	0778787879		
3	Mathara	0787878789		
2	Nugegoda	0322222221		

Explanation:

- This code checks if the HTTP request method is POST and if specific form fields (e.g., "branchName," "branchIdentification," and "phoneNumber") have been submitted.
- It retrieves data from the form fields using `$_POST` superglobal.
- The SQL `INSERT` statement inserts the data into the "Branch" table.
- Prepared statements are used for SQL queries to prevent SQL injection.
- The `bind_param` method binds the form data to placeholders in the SQL query.
- If the query execution is successful, it displays a success message; otherwise, it shows an error message.

2. Read (Retrieve) Operation:

The "Read" operation involves retrieving and displaying branch records from the "Branch" table.



```
branch.php
<?php
include 'db.php'; // Include your database connection file

// Select all records from the Branch table
$sql = "SELECT * FROM Branch";

$result = $conn->query($sql);

if ($result->num_rows > 0) {
    while ($row = $result->fetch_assoc()) {
        echo "<tr>";
        echo "<td>" . $row["BranchID"] . "</td>";
        echo "<td>" . $row["BranchName"] . "</td>";
        echo "<td>" . $row["PhoneNumber"] . "</td>";
        echo "<td><a href='branch-edit-form.php?branchID=' . $row['BranchID'] . '&branchName=' . $row['BranchName'] . '&phoneNumb";
        echo "<td><a href='branch-delete.php?branchID=' . $row['BranchID'] . '&branchName=' . $row['BranchName'] . '><i class='b";
        echo "</tr>";
    }
} else {
    echo "No branches found.";
}

$conn->close();
?>

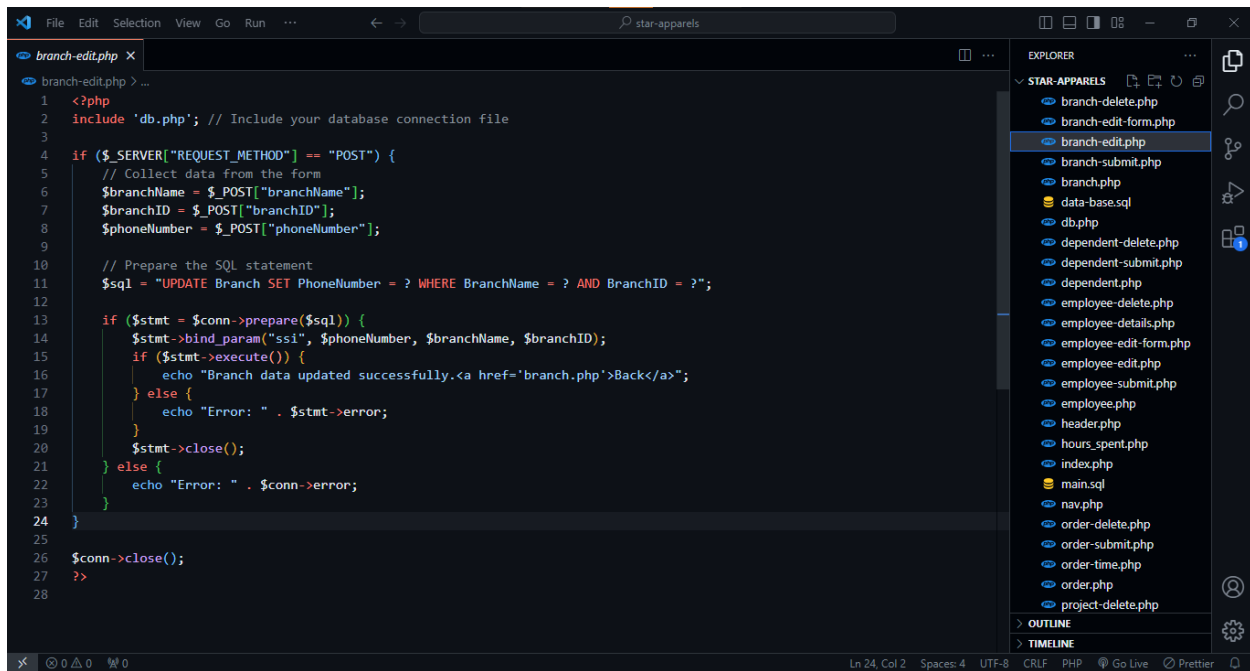
</tbody>
</table>
</div>
```

Explanation:

- The code executes an SQL `SELECT` statement to retrieve all records from the "Branch" table.
- The result is stored in the `$result` variable.
- It checks if there are rows returned (`num_rows > 0`) and iterates through the results to display branch data.
- If no branches are found, it shows a message indicating that no branches were found.

3. Update Operation:

The "Update" operation allows you to modify existing branch records in the "Branch" table.



```
1 <?php
2 include 'db.php'; // Include your database connection file
3
4 if ($_SERVER["REQUEST_METHOD"] == "POST") {
5     // Collect data from the form
6     $branchName = $_POST["branchName"];
7     $branchID = $_POST["branchID"];
8     $phoneNumber = $_POST["phoneNumber"];
9
10    // Prepare the SQL statement
11    $sql = "UPDATE Branch SET PhoneNumber = ? WHERE BranchName = ? AND BranchID = ?";
12
13    if ($stmt = $conn->prepare($sql)) {
14        $stmt->bind_param("ssi", $phoneNumber, $branchName, $branchID);
15        if ($stmt->execute()) {
16            echo "Branch data updated successfully.<a href='branch.php'>Back</a>";
17        } else {
18            echo "Error: " . $stmt->error;
19        }
20        $stmt->close();
21    } else {
22        echo "Error: " . $conn->error;
23    }
24 }
25
26 $conn->close();
27 ?>
28
```

Explanation:

- This code checks if the HTTP request method is POST and if a "branchID" field has been submitted, along with other form fields for updating.
- It retrieves data from the form fields using `$_POST` superglobal.
- The SQL UPDATE statement is used to modify branch data based on the provided "branchID."
- Prepared statements are used for SQL queries to prevent SQL injection.
- The `bind_param` method binds the updated data to placeholders in the SQL query.
- If the update is successful, it displays a success message; otherwise, it shows an error message.

Branches

Input Data

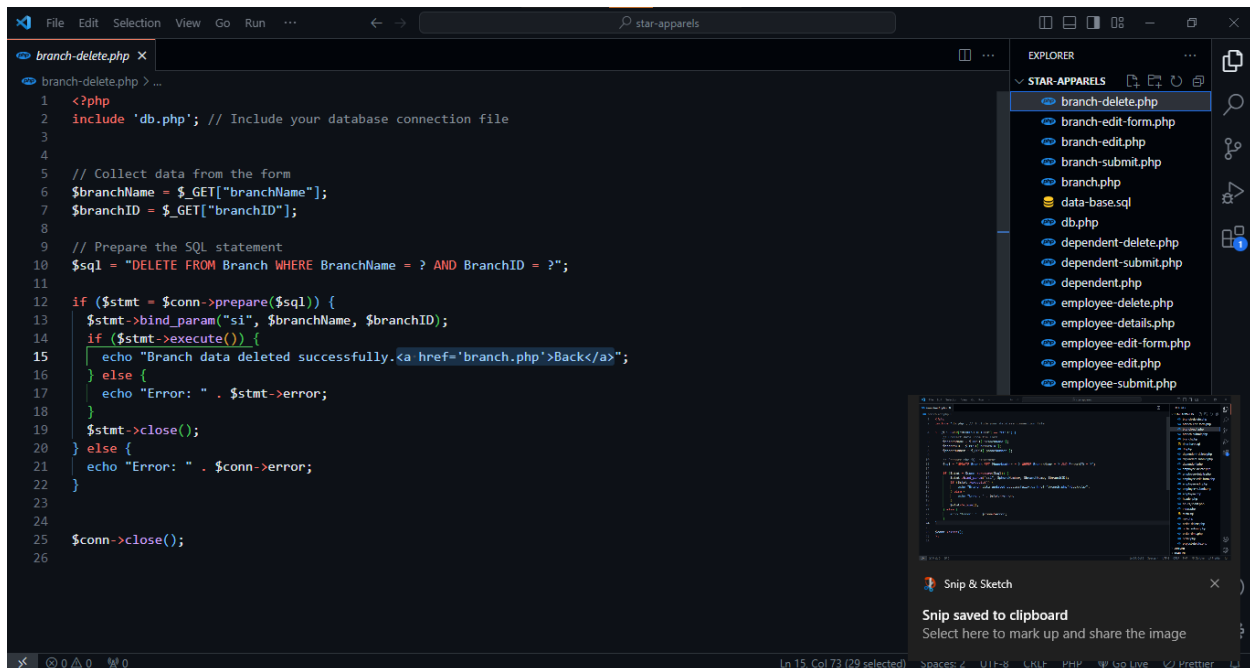
ID	1
Name	Colombo
Phone Number	0778787879

Submit

Branch Update form interface

4. Delete Operation:

The "Delete" operation allows you to remove branch records from the "Branch" table.



```
1 <?php
2 include 'db.php'; // Include your database connection file
3
4
5 // Collect data from the form
6 $branchName = $_GET["branchName"];
7 $branchID = $_GET["branchID"];
8
9 // Prepare the SQL statement
10 $sql = "DELETE FROM Branch WHERE BranchName = ? AND BranchID = ?";
11
12 if ($stmt = $conn->prepare($sql)) {
13     $stmt->bind_param("si", $branchName, $branchID);
14     if ($stmt->execute()) {
15         echo "Branch data deleted successfully.<a href='branch.php'>Back</a>";
16     } else {
17         echo "Error: " . $stmt->error;
18     }
19     $stmt->close();
20 } else {
21     echo "Error: " . $conn->error;
22 }
23
24
25 $conn->close();
26
```

Explanation:

- This code checks if the HTTP request method is POST and if a "branchIDToDelete" field has been submitted.
- It retrieves the branch ID to be deleted using `$_POST` superglobal.
- The SQL `DELETE` statement is used to remove the branch record based on the provided "branchID."
- Prepared statements are used for SQL queries to prevent SQL injection.
- If the deletion is successful, it displays a success message; otherwise, it shows an error message.

In all operations, data is retrieved from HTML forms using `$_POST` based on the form field names. Prepared statements are used for SQL queries to prevent SQL injection, and appropriate success or error messages are displayed to inform the user of the operation's result.

Feature we Implemented

1. Employee Management:

- **Employee Records:** The system allows you to create and maintain detailed records for each employee, including personal information, contact details, salary, and date of birth.
- **Branch Assignment:** Employees can be assigned to specific branches within the organization, and this information is tracked.
- **Dependents:** The system keeps track of an employee's dependents, including their names and dates of birth. It enforces a maximum limit of three dependents per employee.

2. Branch Management:

- **Branch Information:** You can add, update, and delete branch records, including branch names, identification numbers, and phone numbers.
- **Employee Assignment:** Each branch can have multiple employees assigned to it. The system tracks which employees work at each branch.

3. Order and Project Management:

- **Order and Project Tracking:** The system allows for the creation and management of orders and projects, each with a unique identifier. You can assign employees to orders and projects, and it tracks how many hours each employee spends on a specific order or project.
- **Supervisor Roles:** Only supervisors of orders or projects have access to view the hours spent by employees on those specific tasks.

4. Dependent Management:

- **Dependent Information:** You can add, update, and delete information about dependents associated with employees, including their names, dates of birth, and relationships to the employees.

5. User-Friendly Interfaces:

- **HTML Forms:** The project includes HTML forms for adding, updating, and deleting records for employees, branches, orders, projects, and dependents. These forms are designed with Bootstrap for a user-friendly and responsive interface.

6. Data Validation and Security:

- **Data Validation:** The system enforces data validation to ensure that only valid data is entered. It checks for proper formats, such as valid phone numbers and dates.
- **Security Measures:** Security features are implemented to protect the system from common threats, including SQL injection. Prepared statements are used to prevent SQL injection attacks.

7. Database Design and UML Notation:

- **Database Schema:** The project follows a well-designed database schema with appropriate tables and relationships.
- **UML Notation:** A UML class diagram is created to visually represent the database schema, tables, and their relationships, using rectangles for tables and lines with arrows for relationships.

8. CRUD Operations:

- **Create, Read, Update, delete (CRUD):** The system implements CRUD operations for managing records in the database, including adding, viewing, updating, and deleting employee, branch, order, project, and dependent records.

Interface and Forms

The screenshot displays the 'STAR APPARELS' Home page. At the top, there is a navigation bar with links: Home, Branch, Employee, Order, Project, and Dependent. The main content area is divided into two columns.

Left Column:

- Employees:** A list of employee names: Jhons and Sumanaweera.
- Employees Work hours Order:** A table with two rows of data:

Order ID: 1	Employee ID: 1	HoursSpent: 2.00
Order ID: 1	Employee ID: 1	HoursSpent: 3.00
- Employees Work hours Projects:** A table with one row of data:

Order ID: 1	Employee ID: 1	HoursSpent: 2.00
-------------	----------------	------------------

Right Column:

- Working Hours Orders:** A form with three input fields: 'Select Employee:' (Jhons), 'Select Order:' (SDE), and 'Work Hours:'. Below these fields is a green 'Add Dependent' button.
- Working Hours Projects:** A form with three input fields: 'Select Employee:' (Jhons), 'Select Order:' (ES2), and 'Work Hours:'. Below these fields is a green 'Add Dependent' button.

Home Page showing Employee names and work hours of Employees. Also Admins can add work hours to employees.

STAR APPARELS

HomeBranchEmployeeOrderProjectDependent

Employee

Input Data

Select Employee:

Jhons

Dependent ID:

Dependent Name:

Dependent Date of Birth:

yyyy-mm-dd

Relationship:

Coverage:

Add Dependent

All data

Dependent ID	Employee Name	Dependent Name	Date of Birth	Relationship	Coverage	Edit
0	Jhons	Nike	2023-09-14	Son	50000	
1	Jhons	Kathy	2023-09-08	Wife	20000	

Employees CRUD interface.

STAR APPARELS

HomeBranchEmployeeOrderProjectDependent

Employee Details Page

Jhons

Employee ID: 1

Residence: Jaffna

Salary: 32000.00

Date of Birth: 2023-02-15

Dependent not found.

Employees single details page

STAR APPARELS

HomeBranchEmployeeOrderProjectDependent

Project

Input Data

Project ID:

Client Name:

Add Order

All data

Project ID	Project Name	Delete
1	ES2	
2	EC2	
3	RR1	

Project Details Page

STAR APPARELS

HomeBranchEmployeeOrderProjectDependent

Order

Input Data

Order ID:

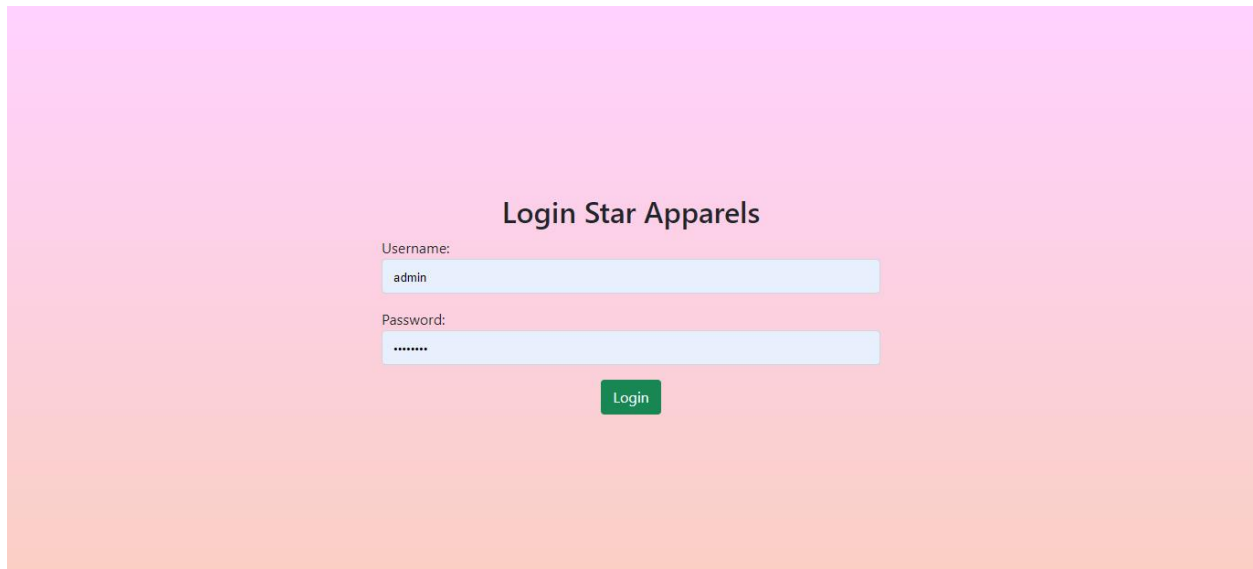
Client Name:

Add Order

All data

Order ID	Client Name	Delete
1	SDE	
2	GER	
3	VE1	

Order Details Page

A login page with a pink-to-orange gradient background. The title "Login Star Apparels" is centered at the top. Below it, the "Username:" label is followed by a light blue input field containing the text "admin". The "Password:" label is followed by a light blue input field containing seven dots. A green "Login" button is centered below the password field.

Login Star Apparels

Username:
admin

Password:

Login

Login Page

GROUP MEMBERS

H.A.H.T.De Silva - D/DBA/23/0027

K.K. Komudi Gimhani - D/DBA/23/0021

W.D.K Shihara - D/DBA/23/0032

S.D.V.Sithum Dayananda - D/DBA/23/0036